JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class System

java.lang.Object
    java.lang.System

```
public final class System
extends Object
```

The System class contains several useful class fields and methods. It cannot be instantiated. Among the facilities provided by the System class are standard input, standard output, and error output streams; access to externally defined properties and environment variables; a means of loading files and libraries; and a utility method for quickly copying a portion of an array.

Since:

1.0

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static interface	System.Logger	System.Logger instances log messages that will be routed to the underlying logging framework the LoggerFinder uses.
static class	System.LoggerFinder	The LoggerFinder service is responsible for creating, managing, and configuring loggers to the underlying framework it uses.

Field Summary

Fields

Modifier and Type	Field	Description
static final PrintStream	err	The "standard" error output stream.
static final InputStream	in	The "standard" input stream.

static final PrintStream	**out**	The "standard" output stream.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
static void	**arraycopy**(Object src, int srcPos, Object dest, int destPos, int length)	Copies an array from the specified source array, beginning at the specified position, to the specified position of the destination array.
static String	**clearProperty**(String key)	Removes the system property indicated by the specified key.
static Console	**console**()	Returns the unique Console object associated with the current Java virtual machine, if any.
static long	**currentTimeMillis**()	Returns the current time in milliseconds.
static void	**exit**(int status)	Initiates the shutdown sequence of the Java Virtual Machine.
static void	**gc**()	Runs the garbage collector in the Java Virtual Machine.
static Map<String, String>	**getenv**()	Returns an unmodifiable string map view of the current system environment.
static String	**getenv**(String name)	Gets the value of the specified environment variable.

static System.Logger	**getLogger**(String name)	Returns an instance of Logger for the caller's use.
static System.Logger	**getLogger**(String name, ResourceBundle bundle)	Returns a localizable instance of Logger for the caller's use.
static Properties	**getProperties**()	Determines the current system properties.
static String	**getProperty**(String key)	Gets the system property indicated by the specified key.
static String	**getProperty**(String key, String def)	Gets the system property indicated by the specified key.
static SecurityManager	**getSecurityManager**()	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally returned the system-wide Security Manager.
static int	**identityHashCode**(Object x)	Returns the same hash code for the given object as would be returned by the default method hashCode(), whether or not the given object's class overrides hashCode().
static Channel	**inheritedChannel**()	Returns the channel inherited from the entity that created this Java virtual machine.
static String	**lineSeparator**()	Returns the system-dependent line separator string.

static void	**load**(String filename)	**Restricted.** Loads the native library specified by the filename argument.
static void	**loadLibrary**(String libname)	**Restricted.** Loads the native library specified by the `libname` argument.
static String	**mapLibraryName**(String libname)	Maps a library name into a platform-specific string representing a native library.
static long	**nanoTime**()	Returns the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds.
static void	**runFinalization**()	**Deprecated, for removal: This API element is subject to removal in a future version.** Finalization has been deprecated for removal.
static void	**setErr**(PrintStream err)	Reassigns the "standard" error output stream.
static void	**setIn**(InputStream in)	Reassigns the "standard" input stream.
static void	**setOut**(PrintStream out)	Reassigns the "standard" output stream.
static void	**setProperties**(Properties props)	Sets the system properties to the `Properties` argument.
static String	**setProperty**(String key, String value)	Sets the system property indicated by the specified key.

static void	**setSecurityManager**(SecurityManager sm)	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally set the system-wide Security Manager.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

in

```
public static final InputStream in
```

The "standard" input stream. This stream is already open and ready to supply input data. This stream corresponds to keyboard input or another input source specified by the host environment or user. Applications should use the encoding specified by the `stdin.encoding` property to convert input bytes to character data.

API Note:

The typical approach to read character data is to wrap `System.in` within the object that handles character encoding. After this is done, subsequent reading should use only the wrapper object; continuing to operate directly on `System.in` results in unspecified behavior.

Here are two common examples. Using an `InputStreamReader`:

```
new InputStreamReader(System.in, System.getProperty("stdin.encoding"));
```

Or using a `Scanner`:

```
new Scanner(System.in, System.getProperty("stdin.encoding"));
```

For handling interactive input, consider using `Console`.

See Also:

Console, stdin.encoding

out

`public static final PrintStream out`

The "standard" output stream. This stream is already open and ready to accept output data. Typically this stream corresponds to display output or another output destination specified by the host environment or user. The encoding used in the conversion from characters to bytes is equivalent to `stdout.encoding`.

For simple stand-alone Java applications, a typical way to write a line of output data is:

```
    System.out.println(data)
```

See the `println` methods in class `PrintStream`.

See Also:

`PrintStream.println()`,
`PrintStream.println(boolean)`,
`PrintStream.println(char)`,
`PrintStream.println(char[])`,
`PrintStream.println(double)`,
`PrintStream.println(float)`,
`PrintStream.println(int)`,
`PrintStream.println(long)`,
`PrintStream.println(java.lang.Object)`,
`PrintStream.println(java.lang.String)`,
stdout.encoding

err

`public static final PrintStream err`

The "standard" error output stream. This stream is already open and ready to accept output data.

Typically this stream corresponds to display output or another output destination specified by the host environment or user. By convention, this output stream is used to display error messages or other information that should come to the immediate attention of a user even if the principal output stream, the value of the variable `out`, has been redirected to a file or other destination that is typically not continuously monitored. The encoding used in the conversion from characters to bytes is equivalent to `stderr.encoding`.

See Also:

stderr.encoding

Method Details

setIn

`public static void setIn(InputStream in)`

Reassigns the "standard" input stream.

Parameters:

`in` - the new standard input stream.

Since:

1.1

setOut

`public static void setOut(PrintStream out)`

Reassigns the "standard" output stream.

Parameters:

`out` - the new standard output stream

Since:

1.1

setErr

`public static void setErr(PrintStream err)`

Reassigns the "standard" error output stream.

Parameters:

`err` - the new standard error output stream.

Since:

1.1

console

`public static Console console()`

Returns the unique `Console` object associated with the current Java virtual machine, if any.

Returns:

The system console, if any, otherwise `null`.

Since:

1.6

See Also:

`Console`

inheritedChannel

```
public static Channel inheritedChannel()
                                 throws IOException
```

Returns the channel inherited from the entity that created this Java virtual machine. This method returns the channel obtained by invoking the `inheritedChannel` method of the system-wide default `SelectorProvider` object.

In addition to the network-oriented channels described in `inheritedChannel`, this method may return other kinds of channels in the future.

Returns:

The inherited channel, if any, otherwise `null`.

Throws:

`IOException` - If an I/O error occurs

Since:

1.5

setSecurityManager

```
@Deprecated(since="17",
            forRemoval=true)
public static void setSecurityManager(SecurityManager sm)
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally set the system-wide Security Manager. Setting a Security Manager is no longer supported. There is no replacement for the Security Manager or this method.

Throws `UnsupportedOperationException`. Setting a security manager is not supported.

Parameters:

`sm` - ignored

Throws:

`UnsupportedOperationException` - always

See Also:

`getSecurityManager()`

getSecurityManager

```
@Deprecated(since="17",
            forRemoval=true)
public static SecurityManager getSecurityManager()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally returned the system-wide Security Manager. Setting a Security Manager is no longer supported. There is no replacement for the Security Manager or this method.

Returns `null`. Setting a security manager is not supported.

Returns:

`null`

See Also:

`setSecurityManager(SecurityManager)`

currentTimeMillis

```
public static long currentTimeMillis()
```

Returns the current time in milliseconds. Note that while the unit of time of the return value is a millisecond, the granularity of the value depends on the underlying operating system and may be larger. For example, many operating systems measure time in units of tens of milliseconds.

See the description of the class `Date` for a discussion of slight discrepancies that may arise between "computer time" and coordinated universal time (UTC).

Returns:

the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.

See Also:

`Date`

nanoTime

```
public static long nanoTime()
```

Returns the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds. This method can only be used to

measure elapsed time and is not related to any other notion of system or wall-clock time. The value returned represents nanoseconds since some fixed but arbitrary *origin* time (perhaps in the future, so values may be negative). The same origin is used by all invocations of this method in an instance of a Java virtual machine; other virtual machine instances are likely to use a different origin.

This method provides nanosecond precision, but not necessarily nanosecond resolution (that is, how frequently the value changes) - no guarantees are made except that the resolution is at least as good as that of `currentTimeMillis()`.

Differences in successive calls that span greater than approximately 292 years (2^{63} nanoseconds) will not correctly compute elapsed time due to numerical overflow.

The values returned by this method become meaningful only when the difference between two such values, obtained within the same instance of a Java virtual machine, is computed.

For example, to measure how long some code takes to execute:

```
long startTime = System.nanoTime();
// ... the code being measured ...
long elapsedNanos = System.nanoTime() - startTime;
```

To compare elapsed time against a timeout, use

```
if (System.nanoTime() - startTime >= timeoutNanos) ...
```

instead of

```
if (System.nanoTime() >= startTime + timeoutNanos) ...
```

because of the possibility of numerical overflow.

Returns:

the current value of the running Java Virtual Machine's high-resolution time source, in nanoseconds

Since:

1.5

arraycopy

```
public static void arraycopy(Object src,
                             int srcPos,
                             Object dest,
                             int destPos,
                             int length)
```

Copies an array from the specified source array, beginning at the specified position, to the specified position of the destination array. A subsequence of array components are copied from the source array referenced by `src` to the destination array referenced by `dest`. The number of components copied is equal to the `length` argument. The components at positions `srcPos` through `srcPos+length-1` in the source array are copied into positions `destPos` through `destPos+length-1`, respectively, of the destination array.

If the `src` and `dest` arguments refer to the same array object, then the copying is performed as if the components at positions `srcPos` through `srcPos+length-1` were first copied to a temporary array with `length` components and then the contents of the temporary array were copied into positions `destPos` through `destPos+length-1` of the destination array.

If `dest` is `null`, then a `NullPointerException` is thrown.

If `src` is `null`, then a `NullPointerException` is thrown and the destination array is not modified.

Otherwise, if any of the following is true, an `ArrayStoreException` is thrown and the destination is not modified:

- The `src` argument refers to an object that is not an array.
- The `dest` argument refers to an object that is not an array.
- The `src` argument and `dest` argument refer to arrays whose component types are different primitive types.
- The `src` argument refers to an array with a primitive component type and the `dest` argument refers to an array with a reference component type.
- The `src` argument refers to an array with a reference component type and the `dest` argument refers to an array with a primitive component type.

Otherwise, if any of the following is true, an `IndexOutOfBoundsException` is thrown and the destination is not modified:

- The `srcPos` argument is negative.
- The `destPos` argument is negative.
- The `length` argument is negative.

- `srcPos+length` is greater than `src.length`, the length of the source array.
- `destPos+length` is greater than `dest.length`, the length of the destination array.

Otherwise, if any actual component of the source array from position `srcPos` through `srcPos+length-1` cannot be converted to the component type of the destination array by assignment conversion, an `ArrayStoreException` is thrown. In this case, let ***k*** be the smallest nonnegative integer less than length such that `src[srcPos+`*k*`]` cannot be converted to the component type of the destination array; when the exception is thrown, source array components from positions `srcPos` through `srcPos+`*k*`-1` will already have been copied to destination array positions `destPos` through `destPos+`*k*`-1` and no other positions of the destination array will have been modified. (Because of the restrictions already itemized, this paragraph effectively applies only to the situation where both arrays have component types that are reference types.)

Parameters:

`src` - the source array.

`srcPos` - starting position in the source array.

`dest` - the destination array.

`destPos` - starting position in the destination data.

`length` - the number of array elements to be copied.

Throws:

`IndexOutOfBoundsException` - if copying would cause access of data outside array bounds.

`ArrayStoreException` - if an element in the `src` array could not be stored into the `dest` array because of a type mismatch.

`NullPointerException` - if either `src` or `dest` is `null`.

identityHashCode

```
public static int identityHashCode(Object x)
```

Returns the same hash code for the given object as would be returned by the default method hashCode(), whether or not the given object's class overrides hashCode(). The hash code for the null reference is zero.

Parameters:

`x` - object for which the hashCode is to be calculated

Returns:

the hashCode

Since:

1.1

See Also:

`Object.hashCode()`, `Objects.hashCode(Object)`

getProperties

```
public static Properties getProperties()
```

Determines the current system properties.

The current set of system properties for use by the `getProperty(String)` method is returned as a `Properties` object. If there is no current set of system properties, a set of system properties is first created and initialized. This set of system properties includes a value for each of the following keys unless the description of the associated value indicates that the value is optional.

Key	Description of Associated Value
`java.version`	Java Runtime Environment version, which may be interpreted as a `Runtime.Version`
`java.version.date`	Java Runtime Environment version date, in ISO-8601 YYYY-MM-DD format, which may be interpreted as a `LocalDate`
`java.vendor`	Java Runtime Environment vendor
`java.vendor.url`	Java vendor URL
`java.vendor.version`	Java vendor version *(optional)*
`java.home`	Java installation directory
`java.vm.specification.version`	Java Virtual Machine specification version, whose value is the feature element of the runtime version
`java.vm.specification.vendor`	Java Virtual Machine specification vendor
`java.vm.specification.name`	Java Virtual Machine specification name
`java.vm.version`	Java Virtual Machine implementation version which may be interpreted as a `Runtime.Version`
`java.vm.vendor`	Java Virtual Machine implementation vendor
`java.vm.name`	Java Virtual Machine implementation name

Key	Description of Associated Value
`java.specification.version`	Java Runtime Environment specification version, whose value is the feature element of the runtime version
`java.specification.maintenance.version`	Java Runtime Environment specification maintenance version, may be interpreted as a positive integer *(optional, see below)*
`java.specification.vendor`	Java Runtime Environment specification vendor
`java.specification.name`	Java Runtime Environment specification name
`java.class.version`	Latest Java class file format version recognized by the Java runtime as `"MAJOR.MINOR"` where `MAJOR` and `MINOR` are both formatted as decimal integers
`java.class.path`	Java class path (refer to `ClassLoader.getSystemClassLoader()` for details)
`java.library.path`	List of paths to search when loading libraries
`java.io.tmpdir`	Default temp file path
`os.name`	Operating system name
`os.arch`	Operating system architecture
`os.version`	Operating system version
`file.separator`	File separator ("/" on UNIX)
`path.separator`	Path separator (":" on UNIX)
`line.separator`	Line separator ("\n" on UNIX)
`user.name`	User's account name
`user.home`	User's home directory
`user.dir`	User's current working directory
`native.encoding`	Character encoding name derived from the host environment and the user's settings. Setting this system property on the command line has no effect.
`stdin.encoding`	Character encoding name for `System.in`. The Java runtime can be started with the system property set to `UTF-8`.

Key	Description of Associated Value
	Starting it with the property set to another value results in unspecified behavior.
`stdout.encoding`	Character encoding name for `System.out` and `System.console()`. The Java runtime can be started with the system property set to `UTF-8`. Starting it with the property set to another value results in unspecified behavior.
`stderr.encoding`	Character encoding name for `System.err`. The Java runtime can be started with the system property set to `UTF-8`. Starting it with the property set to another value results in unspecified behavior.

The `java.specification.maintenance.version` property is defined if the specification implemented by this runtime at the time of its construction had undergone a maintenance release. When defined, its value identifies that maintenance release. To indicate the first maintenance release this property will have the value `"1"`, to indicate the second maintenance release this property will have the value `"2"`, and so on.

Multiple paths in a system property value are separated by the path separator character of the platform.

Additional locale-related system properties defined by the `Default Locale` section in the `Locale` class description may also be obtained with this method.

API Note:

Changing a standard system property may have unpredictable results unless otherwise specified. Property values may be cached during initialization or on first use. Setting a standard property after initialization using `getProperties()`, `setProperties(Properties)`, `setProperty(String, String)`, or `clearProperty(String)` may not have the desired effect.

Implementation Note:

In addition to the standard system properties, the system properties may include the following keys:

Key	Description of Associated Value
`jdk.module.path`	The application module path

Key	Description of Associated Value
`jdk.module.upgrade.path`	The upgrade module path
`jdk.module.main`	The module name of the initial/main module
`jdk.module.main.class`	The main class name of the initial module
`file.encoding`	The name of the default charset, defaults to `UTF-8`. The property may be set on the command line to the value `UTF-8` or `COMPAT`. If set on the command line to the value `COMPAT` then the value is replaced with the value of the `native.encoding` property during startup. Setting the property to a value other than `UTF-8` or `COMPAT` results in unspecified behavior.

Returns:

the system properties

See Also:

`setProperties(Properties)`, `Properties`

lineSeparator

```
public static String lineSeparator()
```

Returns the system-dependent line separator string. It always returns the same value - the initial value of the system property `line.separator`.

On UNIX systems, it returns `"\n"`; on Microsoft Windows systems it returns `"\r\n"`.

Returns:

the system-dependent line separator string

Since:

1.7

setProperties

```
public static void setProperties(Properties props)
```

Sets the system properties to the `Properties` argument.

The argument becomes the current set of system properties for use by the `getProperty(String)` method. If the argument is `null`, then the current set of system properties is forgotten.

API Note:

Changing a standard system property may have unpredictable results unless otherwise specified. See getProperties for details.

Parameters:

`props` - the new system properties.

See Also:

`getProperties()`, `Properties`

getProperty

```
public static String getProperty(String key)
```

Gets the system property indicated by the specified key.

If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the `getProperties` method.

API Note:

Changing a standard system property may have unpredictable results unless otherwise specified. See getProperties for details.

Parameters:

`key` - the name of the system property.

Returns:

the string value of the system property, or `null` if there is no property with that key.

Throws:

`NullPointerException` - if `key` is `null`.

`IllegalArgumentException` - if `key` is empty.

See Also:

`setProperty(String, String)`,
`getProperties()`

getProperty

```
public static String getProperty(String key,
                                 String def)
```

Gets the system property indicated by the specified key.

If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the `getProperties` method.

Parameters:

key - the name of the system property.

def - a default value.

Returns:

the string value of the system property, or the default value if there is no property with that key.

Throws:

NullPointerException - if key is null.

IllegalArgumentException - if key is empty.

See Also:

setProperty(String, String),
getProperties()

setProperty

```
public static String setProperty(String key,
                                 String value)
```

Sets the system property indicated by the specified key.

API Note:

Changing a standard system property may have unpredictable results unless otherwise specified. See getProperties for details.

Parameters:

key - the name of the system property.

value - the value of the system property.

Returns:

the previous value of the system property, or null if it did not have one.

Throws:

NullPointerException - if key or value is null.

IllegalArgumentException - if key is empty.

Since:

1.2

See Also:

getProperty(String),
getProperty(java.lang.String),
getProperty(java.lang.String, java.lang.String)

clearProperty

```
public static String clearProperty(String key)
```

Removes the system property indicated by the specified key.

API Note:

Changing a standard system property may have unpredictable results unless otherwise specified. See getProperties method for details.

Parameters:

key - the name of the system property to be removed.

Returns:

the previous string value of the system property, or `null` if there was no property with that key.

Throws:

`NullPointerException` - if key is `null`.

`IllegalArgumentException` - if key is empty.

Since:

1.5

See Also:

`getProperty(String)`,
`setProperty(String, String)`,
`Properties`

getenv

```
public static String getenv(String name)
```

Gets the value of the specified environment variable. An environment variable is a system-dependent external named value.

System properties and *environment variables* are both conceptually mappings between names and values. Both mechanisms can be used to pass user-defined information to a Java process. Environment variables have a more global effect, because they are visible to all descendants of the process which defines them, not just the immediate Java subprocess. They can have subtly different semantics, such as case insensitivity, on different operating systems. For these reasons, environment variables are more likely to have unintended side effects. It is best to use system properties where possible. Environment variables should be used when a global effect is desired, or when an external system interface requires an environment variable (such as `PATH`).

On UNIX systems the alphabetic case of `name` is typically significant, while on Microsoft Windows systems it is typically not. For example, the expression `System.getenv("FOO").equals(System.getenv("foo"))` is likely to be true on Microsoft Windows.

Parameters:

name - the name of the environment variable

Returns:

the string value of the variable, or `null` if the variable is not defined in the system environment

Throws:

`NullPointerException` - if `name` is `null`

See Also:

`getenv()`, `ProcessBuilder.environment()`

getenv

```
public static Map<String,String> getenv()
```

Returns an unmodifiable string map view of the current system environment. The environment is a system-dependent mapping from names to values which is passed from parent to child processes.

If the system does not support environment variables, an empty map is returned.

The returned map will never contain null keys or values. Attempting to query the presence of a null key or value will throw a `NullPointerException`. Attempting to query the presence of a key or value which is not of type `String` will throw a `ClassCastException`.

The returned map and its collection views may not obey the general contract of the `Object.equals(Object)` and `Object.hashCode()` methods.

The returned map is typically case-sensitive on all platforms.

When passing information to a Java subprocess, system properties are generally preferred over environment variables.

Returns:

the environment as a map of variable names to values

Since:

1.5

See Also:

`getenv(String)`, `ProcessBuilder.environment()`

getLogger

```
public static System.Logger getLogger(String name)
```

Returns an instance of `Logger` for the caller's use.

API Note:

This method may defer calling the `LoggerFinder.getLogger` method to create an actual logger supplied by the logging backend, for instance, to allow loggers to be obtained during the system initialization time.

Implementation Requirements:

Instances returned by this method route messages to loggers obtained by calling `LoggerFinder.getLogger(name, module)`, where `module` is the caller's module. In cases where `System.getLogger` is called from a context where there is no caller frame on the stack (e.g when called directly from a JNI attached thread), `IllegalCallerException` is thrown. To obtain a logger in such a context, use an auxiliary class that will implicitly be identified as the caller, or use the system `LoggerFinder` to obtain a logger instead. Note that doing the latter may eagerly initialize the underlying logging system.

Parameters:

`name` - the name of the logger.

Returns:

an instance of `System.Logger` that can be used by the calling class.

Throws:

`NullPointerException` - if `name` is `null`.

`IllegalCallerException` - if there is no Java caller frame on the stack.

Since:

9

getLogger

```
public static System.Logger getLogger(String name,
                                       ResourceBundle bundle)
```

Returns a localizable instance of `Logger` for the caller's use. The returned logger will use the provided resource bundle for message localization.

API Note:

This method is intended to be used after the system is fully initialized. This method may trigger the immediate loading and initialization of the `System.LoggerFinder` service, which may cause issues if the Java Runtime is not ready to initialize the concrete service implementation yet. System classes which may be loaded early in the boot sequence and need to log localized messages should create a logger using `getLogger(java.lang.String)` and then use the log methods that take a resource bundle as parameter.

Implementation Requirements:

The returned logger will perform message localization as specified by `LoggerFinder.getLocalizedLogger(name, bundle, module)`, where `module` is the caller's module. In cases where `System.getLogger` is called from a context where there is no caller frame on the stack (e.g when called directly from a JNI attached thread), `IllegalCallerException` is thrown. To obtain a logger in such a context, use an auxiliary class that will implicitly be identified as the caller, or use the system `LoggerFinder` to obtain a logger instead. Note that doing the latter may eagerly initialize the underlying logging system.

Parameters:

`name` - the name of the logger.

`bundle` - a resource bundle.

Returns:

an instance of `System.Logger` which will use the provided resource bundle for message localization.

Throws:

`NullPointerException` - if `name` is `null` or `bundle` is `null`.

`IllegalCallerException` - if there is no Java caller frame on the stack.

Since:

9

exit

```
public static void exit(int status)
```

Initiates the shutdown sequence of the Java Virtual Machine. This method initiates the shutdown sequence (if it is not already initiated) and then blocks indefinitely. This method neither returns nor throws an exception; that is, it does not complete either normally or abruptly.

The argument serves as a status code. By convention, a nonzero status code indicates abnormal termination.

The call `System.exit(n)` is effectively equivalent to the call:

```
    Runtime.getRuntime().exit(n)
```

Implementation Note:

The initiation of the shutdown sequence is logged by `Runtime.exit(int)`.

Parameters:

`status` - exit status.

See Also:

`Runtime.exit(int)`

gc

```
public static void gc()
```

Runs the garbage collector in the Java Virtual Machine.

Calling the `gc` method suggests that the Java Virtual Machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for reuse by the Java Virtual Machine. When control returns from the method call, the Java Virtual Machine has made a best effort to reclaim space from all unused objects. There is no guarantee that this effort will recycle any particular number of unused objects, reclaim any particular amount of space, or complete at any particular time, if at all, before the method returns or ever. There is also no guarantee that this effort will determine the change of reachability in any particular number of objects, or that any particular number of `Reference` objects will be cleared and enqueued.

The call `System.gc()` is effectively equivalent to the call:

```
Runtime.getRuntime().gc()
```

See Also:

`Runtime.gc()`

runFinalization

```
@Deprecated(since="18",
            forRemoval=true)
public static void runFinalization()
```

Deprecated, for removal: This API element is subject to removal in a future version.
Finalization has been deprecated for removal. See `Object.finalize()` for background information and details about migration options.

When running in a JVM in which finalization has been disabled or removed, no objects will be pending finalization, so this method does nothing.

Runs the finalization methods of any objects pending finalization. Calling this method suggests that the Java Virtual Machine expend effort toward running the `finalize` methods of objects that have been found to be discarded but whose `finalize` methods have not yet been run. When control returns from

the method call, the Java Virtual Machine has made a best effort to complete all outstanding finalizations.

The call `System.runFinalization()` is effectively equivalent to the call:

```
Runtime.getRuntime().runFinalization()
```

See *Java Language Specification*:

12.6 Finalization of Class Instances

See Also:

`Runtime.runFinalization()`

load

```
public static void load(String filename)
```

> **`load` is a restricted method of the Java platform.**
> *Programs can only use `load` when access to restricted methods is enabled. Restricted methods are unsafe, and, if used incorrectly, might crash the JVM or result in memory corruption.*

Loads the native library specified by the filename argument. The filename argument must be an absolute path name. If the filename argument, when stripped of any platform-specific library prefix, path, and file extension, indicates a library whose name is, for example, L, and a native library called L is statically linked with the VM, then the JNI_OnLoad_L function exported by the library is invoked rather than attempting to load a dynamic library. A filename matching the argument does not have to exist in the file system. See the JNI Specification for more details. Otherwise, the filename argument is mapped to a native library image in an implementation-dependent manner.

The call `System.load(name)` is effectively equivalent to the call:

```
Runtime.getRuntime().load(name)
```

Parameters:

`filename` - the file to load.

Throws:

`UnsatisfiedLinkError` - if either the filename is not an absolute path name, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.

`NullPointerException` - if `filename` is `null`

`IllegalCallerException` - if the caller is in a module that does not have native access enabled.

External Specifications

Java Native Interface Specification

See Also:

`Runtime.load(java.lang.String)` RESTRICTED

loadLibrary

```
public static void loadLibrary(String libname)
```

`loadLibrary` is a restricted method of the Java platform.
Programs can only use `loadLibrary` when access to restricted methods is enabled.
Restricted methods are unsafe, and, if used incorrectly, might crash the JVM or result in memory corruption.

Loads the native library specified by the `libname` argument. The `libname` argument must not contain any platform specific prefix, file extension or path. If a native library called `libname` is statically linked with the VM, then the JNI_OnLoad_`libname` function exported by the library is invoked. See the JNI Specification for more details. Otherwise, the libname argument is loaded from a system library location and mapped to a native library image in an implementation-dependent manner.

The call `System.loadLibrary(name)` is effectively equivalent to the call

```
Runtime.getRuntime().loadLibrary(name)
```

Parameters:

`libname` - the name of the library.

Throws:

`UnsatisfiedLinkError` - if either the libname argument contains a file path, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.

`NullPointerException` - if `libname` is `null`

`IllegalCallerException` - if the caller is in a module that does not have native access enabled.

External Specifications

Java Native Interface Specification

See Also:

`Runtime.loadLibrary(java.lang.String)` RESTRICTED

mapLibraryName

```
public static String mapLibraryName(String libname)
```

Maps a library name into a platform-specific string representing a native library.

Parameters:

`libname` - the name of the library.

Returns:

a platform-dependent native library name.

Throws:

`NullPointerException` - if `libname` is `null`

Since:

1.2

See Also:

`loadLibrary(java.lang.String)` RESTRICTED,
`ClassLoader.findLibrary(java.lang.String)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.